GRank: Towards Target-Aware and Streamlined Industrial Retrieval with a Generate-Rank Framework

Yijia Sun*
sunyijia@kuaishou.com
Kuaishou Technology
Beijing, China

Shanshan Huang*
huangshanshan@kuaishou.com
Kuaishou Technology
Beijing, China

Zhiyuan Guan†
guanzhiyuan03@kuaishou.com
Kuaishou Technology
Beijing, China

Qiang Luo‡
luoqiang@kuaishou.com
Kuaishou Technology
Beijing, China

Ruiming Tang‡
tangruiming@kuaishou.com
Kuaishou Technology
Beijing, China

Kun Gai
gai.kun@qq.com
Unaffiliated
Beijing, China

Guorui Zhou‡
zhouguorui@kuaishou.com
Kuaishou Technology
Beijing, China

Abstract

Industrial-scale recommender systems rely on a cascade pipeline in which the *retrieval* stage must return a high-recall candidate set from *billions* of items under tight latency. Existing solutions either (i) suffer from limited expressiveness in capturing fine-grained user-item interactions, as seen in decoupled dual-tower architectures that rely on separate encoders, or generative models that lack precise target-aware matching capabilities, or (ii) build *structured indices* (tree, graph, quantization) whose item-centric topologies struggle to incorporate dynamic user preferences and incur prohibitive construction and maintenance costs.

We present GRank, a novel *structured-index-free* retrieval paradigm that seamlessly unifies target-aware learning with user-centric retrieval. Our key innovations include: (1) A *target-aware Generator* trained to perform personalized candidate generation via GPU-accelerated MIPS, eliminating semantic drift and maintenance costs of structured indexing; (2) A *lightweight but powerful Ranker* that performs fine-grained, candidate-specific inference on small subsets; (3) An end-to-end multi-task learning framework that ensures semantic consistency between generation and ranking objectives.

Extensive experiments on two public benchmarks and a billion-item production corpus demonstrate that GRank improves Recall@500 by over 30% and 1.7× the P99 QPS of state-of-the-art tree- and graph-based retrievers.

GRank has been **fully deployed in production** in our recommendation platform since Q2 2025, serving 400 million active users with 99.95% service availability. Online A/B tests confirm significant improvements in core engagement metrics, with Total App Usage Time increasing by 0.160% in the main app and 0.165% in the Lite version.

CCS Concepts

• **Information systems** → **Retrieval models and ranking**; *Personalization.*

Keywords

Industrial-Scale Retrieval, Generate→Rank Workflow, Target-Aware Cross-Attention

ACM Reference Format:
Yijia Sun, Shanshan Huang, Zhiyuan Guan, Qiang Luo, Ruiming Tang, Kun Gai, and Guorui Zhou. 2026. GRank: Towards Target-Aware and Streamlined Industrial Retrieval with a Generate-Rank Framework. In *Proceedings of the ACM Web Conference 2026 (WWW '26), April 13–17, 2026, Dubai, United Arab Emirates.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3774904.3792810

1 Introduction

Recommender systems serve as the bedrock of modern information discovery, typically orchestrating a multi-stage cascade (Retrieval → Pre-ranking → Ranking → Re-ranking) to filter billons of items. Within this pipeline, the **retrieval stage** acts as the initial funnel. Its primary mandate is to efficiently identify a coarse candidate set containing the user's potential interests. The quality of this set dictates the upper bound of the entire system's performance.

To understand the evolution and challenges of retrieval models, it is essential to distinguish between two fundamental paradigms regarding how user preferences are modeled:

1. Target-Agnostic Retrieval: The user is encoded into a fixed representation vector **u** (e.g., via Dual-Tower models [5, 10, 16] or sequential transformers [15, 21]), which is independent of any

*These authors contributed equally to this work.
†This author conducted the research while at Kuaishou Technology. He is now with Shanghai Jiaotong University.
‡Corresponding authors.

WWW '26, Dubai, United Arab Emirates

ACM ISBN 979-8-4007-2307-0/2026/04
https://doi.org/10.1145/3774904.3792810

specific candidate item $\mathbf{v}$. Relevance is computed via a lightweight metric, typically the inner product $s(\mathbf{u}, \mathbf{v}) = \mathbf{u}^\top \mathbf{v}$.

2. Target-Aware Retrieval: The user representation is dynamically refined or computed *conditioned* on the specific target candidate $\mathbf{v}$, formulated as $s(\mathbf{u}, \mathbf{v}) = f_\theta(\mathbf{u}, \mathbf{v})$.

Why is this necessary? User interests are often polysemous and context-dependent. A user's history may contain diverse behaviors (e.g., clicking on both "gaming laptops" and "office chairs"). In a target-agnostic model, these interests are compressed into a single mean vector, often diluting specific intents. In contrast, a target-aware model can perform **candidate-specific refinement**: when scoring a "mouse," the model activates the "laptop" history; when scoring a "desk," it attends to the "chair" history. This ability to discern subtle user intents from noisy history is critical for high-precision retrieval.

While target-aware models theoretically offer superior precision, their deployment is hindered by a prohibitive computational cost: executing a complex interaction function $f_\theta(\mathbf{u}, \mathbf{v})$ for every item in a billion-scale corpus is infeasible.

To balance efficiency and precision, existing retrieval paradigms generally fall into two categories, each facing fundamental limitations:

Paradigm 1: The Standard of Target-Agnostic Retrieval. Currently, the vast majority of industrial systems employ target-agnostic architectures, such as Dual-Encoders [5, 10, 16] or sequential transformers [15, 21]. These models decouple user and item encoding, enabling highly efficient retrieval via Maximum Inner-Product Search (MIPS)

Limitation: The efficiency comes at a cost. By compressing a user's complex, multi-faceted interests into a single fixed vector $\mathbf{u}$ *before* seeing any candidate, these models suffer from the "early fusion bottleneck." They preclude any candidate-specific refinement, often failing to retrieve relevant items that lie in the "long tail" of user interests or requiring an excessively large candidate set to maintain recall.

Paradigm 2: The Complexity of Structured Indices. To bridge the gap between efficiency and target-aware precision, a separate line of research (e.g., TDM [26], JTM [25]) incorporates target-aware scoring into **Structured Indices** (Trees or Graphs). Instead of scanning the full corpus, these methods organize items into a hierarchical structure and formulate retrieval as a beam search—traversing from coarse clusters to fine-grained items.

Limitation: While this enables target-aware pruning, it introduces severe engineering and methodological flaws:

- **Structural Rigidity:** These indices rely on *item-centric* clustering (e.g., grouping items by category). This static structure often misaligns with fluid, dynamic user intents. If a user's intent does not match the pre-defined index path, the retrieval fails irrecoverably.
- **Engineering Overhead:** textcolorblueMaintaining tree/graph indices is operationally expensive. They suffer from structural imbalance leading to unpredictable latency (poor P99), and the complex offline construction process creates a bottleneck that hinders rapid model updates.

The Root Cause and Our Insight. The identified limitations expose a fundamental deadlock in retrieval design: *Efficiency* currently demands target-agnostic MIPS, while *Precision* necessitates rigid structured indices. We argue this stems from a monolithic view that conflates two distinct objectives: **Global Pruning (Recall):** Efficiently narrowing a billion-scale corpus to a coarse candidate set. **Local Distinction (Precision):** Fine-grained intent disambiguation via user-item interaction. Existing structured indices attempt to force-fit both objectives into a single search path, incurring immense engineering overhead. **GRank** breaks this deadlock by decoupling these goals into a dedicated *Generate-then-Rank* framework.

Our Solution: The GRank Framework. We propose GRank, a novel *Generate-then-Rank* paradigm that decouples retrieval into two specialized, learnable stages, thereby eliminating the reliance on structured indices.

- **Stage 1 (Generator): Implicitly Target-Aware Pruning.** Instead of a static dual-tower, we design a generator trained with *target-aware supervision*. Through a novel training paradigm, the generator absorbs the ability to anticipate candidate-specific preferences into the user representation. At inference, it retains the efficiency of standard MIPS to produce a compact candidate set, effectively bridging the gap between agnostic architecture and aware capability.
- **Stage 2 (Ranker): Explicitly Target-Aware Scoring.** A lightweight cross-attention network performs fine-grained interaction modeling on the small subset from Stage 1. This stage handles the heavy lifting of intent disambiguation without the latency burden of full-corpus scanning.

By shifting the complexity of target-aware modeling from *inference-time indexing* to *training-time learning*, GRank breaks the efficiency-precision dichotomy.

Key Contributions:

- **Framework:** We propose a universal two-stage retrieval architecture that seamlessly unifies efficient generation and precise ranking in an end-to-end manner.
- **Methodology:** We introduce a novel training strategy that distills target-aware signals into a MIPS-compatible generator, achieving precision without indexing overhead.
- **Validation:** Extensive experiments show over 30% improvement in Recall@500 and 1.7× QPS gain.

GRank has been deployed in production since Q2 2025, serving 50 billion daily requests with 99.95% availability. Online A/B tests confirm significant gains: **Total App Usage Time** increased by 0.160% in the main app and 0.165% in the Lite version.

2 Related Work

2.1 Efficient yet Target-Agnostic Retrieval Paradigms

Dual-Tower Architectures: Early retrieval systems predominantly employed *dual-tower architectures* (e.g., DSSM [10], YouTube Recommendations [5]), which independently encode users and items into embeddings $\mathbf{u} \in \mathbb{R}^d$ and $\mathbf{v} \in \mathbb{R}^d$, then measure relevance via a simple inner product enabling MIPS on GPUs [2, 12, 14] or specialized ANN libraries [9] with logarithmic query time. However, this design is inherently *target-agnostic*: the same user representation

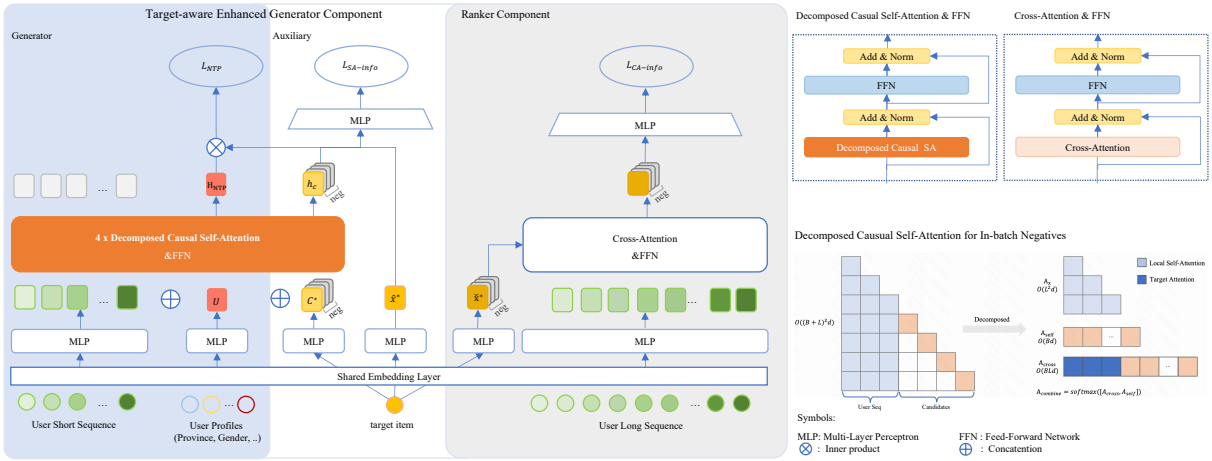


Figure 1: GRank Framework Architecture. The framework comprises two tightly-coupled modules optimized through three complementary losses: (1) Target-Aware Enhanced Generator integrating Generator (optimized by $\mathcal{L}_{\text{NTP}}$) for short-term preference modeling and Auxiliary (optimized by $\mathcal{L}_{\text{SA-info}}$) for target injection; (2) Ranker (optimized by $\mathcal{L}_{\text{CA-info}}$) employing efficient cross-attention for fast inference. The Auxiliary module injects target embedding C^* and in-batch negatives C_j during training only. The Generator utilizes our novel decomposed causal self-attention (detailed in bottom-right) to efficiently process user sequences alongside candidate items while maintaining mathematical equivalence. All components are jointly optimized through multi-task learning.

u is reused for all items, preventing candidate-specific refinement and limiting expressiveness for complex user intents [6].

Generative Retrieval: Recent work reformulates retrieval as a *generative sequence modeling* problem. This paradigm shift began with models like SASRec [15], which employed unidirectional self-attention to model user sequences for next-item prediction. BERT4Rec [21] extended this by introducing bidirectional context encoding, while subsequent works like GPTRec [19] fully embraced an autoregressive framework. Further innovations such as Kuaiformer [17] and MPFormer [22] introduce multi-interest tokens and adaptive sequence compression. Despite their sophistication, these methods remain fundamentally *target-agnostic*: the user representation is still computed independently of the candidate item (i.e., $\text{score}(\mathbf{u}, \mathbf{v}) = \phi(\text{Enc}_{\text{user}}(S), \mathbf{v})$), lacking explicit interaction mechanisms $g(\mathbf{u}, \mathbf{v})$ to focus history on specific candidates [23, 24].

2.2 Target-Aware Retrieval via Structured Indexing

To address the target awareness problem, *index-based retrieval* methods incorporating explicit interaction modeling have emerged. Key approaches include Tree-Based Indexing(such as TDM [26], JTM[25], BSAT[28], MISS[8]), Graph-Based Indexing(NANN [3], DR[7]), and Quantization-Based Indexing(StreamingVQ [1]).

Tree-Based Indexing. Tree-based Deep Models propose tree structures to hierarchically search candidates. **TDM** [26] pioneers tree-based retrieval, reducing complexity to $O(\log N)$. However, its tree hierarchy $\mathcal{T}$ is typically *decoupled* from the model objective $\mathcal{L}_{\text{model}}$. **JTM** [25] alleviates the gap by co-training $\mathcal{T}$ and θ with heuristic clustering. However, the clustering itself is a limitation: user interest $\mathcal{D}_u$ can hardly perfectly aligns with item clusters C_i, and the $O(N \log N)$ index rebuild is prohibitive for billion-scale item corpora C [27].

Graph-Based Indexing. **NANN** [3] resorts to HNSW [18] for candidate searching with model-generated edge weights. However, the construction of $\mathcal{G}$ often relies on pre-defined metrics (e.g. cosine similarity), which impose strong geometric assumptions on the item embedding space. This deviation is exacerbated by non-linear encoders, causing retrieval paths to diverge from ranking objectives [13].

Quantization-Based Indexing. **StreamingVQ** [1] employs quantization clustering to compress high-dimensional embeddings into short codes using a codebook $\mathcal{B} = \{\mathbf{c}_1, ..., \mathbf{c}_K\}$. This enables sub-linear retrieval via table lookups. However, quantization indices typically assume Euclidean space ($\text{sim}(\mathbf{v}, \mathbf{c}_k) = -\|\mathbf{v} - \mathbf{c}_k\|_2^2$). Complex encoders (e.g., those using attention) break this assumption, *exacerbating* the mismatch between the quantization objective $\mathcal{L}_{\text{VQ}}$ and $\mathcal{L}_{\text{rank}}$, further reducing retrieval accuracy [11]. Additionally, training a well-balanced codebook is non-trivial: items often concentrate in a few clusters, diminishing the pruning benefit of table lookups and introducing extra engineering overhead.

Fundamental Limitation: Item-Centric expansion. Beyond these specific technical challenges, all structured indexing methods share a core architectural limitation: *item-centric candidate expansion.* The search path is determined by pre-computed item-item proximities, inherently excluding real-time, personalized user signals during traversal. This results in semantic deviation where retrieval paths diverge from actual user intent [4, 20], as user-specific context cannot dynamically re-route the query.

These limitations motivate our work GRank, which seeks to achieve target-aware precision without relying on structured indices or incurring their associated overheads.

3 Methodology

This section details the **GRank** framework, structured as follows. We begin by formalizing the retrieval task in §3.1. We then present our offline training methodology in §3.2, which jointly optimizes the Generator and Ranker modules end-to-end. Finally, §3.3 describes the efficient two-stage online inference pipeline. Figures 1 and 2 illustrate the complete training and serving architectures, respectively.

3.1 Problem Formulation

Let $\mathcal{S}_u = \{(i_1, f_1), \ldots, (i_n, f_n)\}$ be a user's chronological interaction sequence with items $i_t \in \mathcal{I}$ and features $f_t \in \mathbb{R}^d$ (watch duration, engagement type, author embedding). Given a target item i^*, the retrieval task is to estimating the relevance score $\mathcal{E}(i^* \mid \mathcal{S}_u)$, which defines the engagement probability:

$$p(i^* \mid \mathcal{S}_u) \propto \exp(\mathcal{E}(i^* \mid \mathcal{S}_u)). \tag{1}$$

The objective is to efficiently retrieve the top-K items from corpus $\mathcal{I}$ that maximize this probability.

3.2 Offline Training

GRank consists of two tightly-coupled, end-to-end trainable modules (Figure 1): (1) **Target-Aware Enhanced Generator**, integrating **Generator** for short-term preference modeling and **Auxiliary** for target injection to enhance both Generator and Ranker; (2) a lightweight **Ranker** that performs fine-grained scoring.

3.2.1 Input Representation. All components share a common item embedding layer $\mathbf{E} \in \mathbb{R}^{|\mathcal{I}| \times d}$. A personalized query token $\mathbf{U}$ aggregates user demographics and sum-pooled embeddings from recent behavior sequences (click, long-view, etc.), providing a fixed-size user context:

$$\mathbf{U} = \left[\mathbf{u};\ \mathrm{Sum}(\mathbf{Seq}_{rs});\ \mathrm{Sum}(\mathbf{Seq}_{click});\ \mathrm{Sum}(\mathbf{Seq}_{long\_view})\right],$$

where $\mathbf{u} \in \mathbb{R}^d$ encodes user attributes. Each $\mathbf{Seq}_\cdot$ stacks embeddings of the L_s most recent items of a behavior type. Sum pooling $\mathrm{Sum}(\cdot)$ aggregates each sequence into a fixed-size vector: $\mathrm{Sum}(\mathbf{Seq}_\cdot) = \sum_{i=1}^{L_s} \mathbf{e}_i$.

3.2.2 Target-Aware Enhanced Generator. The generator's core innovation is to *learn target-aware discrimination during training* while maintaining *target-agnostic efficiency during inference*. It comprises:

- **Generator:** A causal Transformer that models short-term user sequences.
- **Auxiliary Module (Training-Only):** Injects target-item and in-batch negative signals into the sequence, enabling the model to learn candidate-aware representations.

Input Encoding & Model Structure. Historical interactions (i_t, f_t) are encoded as $\mathbf{x}_t \in \mathbb{R}^d$. The generator takes the recent history $\mathbf{x}_{L_l - L_s + 1:L_l}$ and the personalized query token $\mathbf{U}$ as input. To inject target-awareness during training, the target item i^* and in-batch negatives $\mathcal{B}$ are transformed via an auxiliary MLP: $\mathbf{C}^* = \mathrm{MLP}_{\mathrm{aux}}(\mathbf{e}^*)$, $\mathbf{C}_j = \mathrm{MLP}_{\mathrm{aux}}(\mathbf{e}_j)$, and appended to form the initial sequence:

$$\mathbf{H}^{(0)} = [\mathbf{x}_{L_l - L_s + 1:L_l};\ \mathbf{U};\ \mathbf{C}], \tag{2}$$

where $\mathbf{C} = [\mathbf{C}^*, \mathbf{C}_1, \ldots, \mathbf{C}_{|\mathcal{B}|}]$. This construction provides target-specific signals and enables efficient InfoNCE loss computation. The subsequent section (§3.2.3) will detail our optimized attention mechanism that efficiently processes this extended sequence while maintaining mathematically equivalent.

The sequence $\mathbf{H}^{(0)}$ is then processed by a causal Transformer decoder, with the user representation $\mathbf{h}_u$ extracted from the output corresponding to $\mathbf{U}$.

Training Objectives. The generator is optimized through two complementary losses:

- **Generator Loss ($\mathcal{L}_{\mathbf{NTP}}$):** The user representation $\mathbf{h}_u$ is normalized by ℓ_2-norm to $\mathbf{H}_{\mathrm{NTP}}$, then contrasted with the target and in-batch negatives $\hat{\mathbf{x}}_j = \mathrm{MLP}_{\mathrm{ntp}}(\mathbf{e}_j)$ using InfoNCE:

$$\mathcal{L}_{\mathrm{NTP}} = -\log \frac{\exp(\langle \mathbf{H}_{\mathrm{NTP}}, \hat{\mathbf{x}}^* \rangle / \tau)}{\sum_{j \in \mathcal{B}} \exp(\langle \mathbf{H}_{\mathrm{NTP}}, \hat{\mathbf{x}}_j \rangle / \tau)}, \tag{3}$$

  where τ is the softmax temperature.
- **Auxiliary Score Loss ($\mathcal{L}_{\mathbf{SA\text{-}info}}$):** The Transformer outputs for the target $\mathbf{h}_{C^*}$ and negatives $\mathbf{h}_{C_j}$ are scored as $s_{\mathrm{sa}_j} = \mathrm{MLP}_{\mathrm{sa}}(\mathbf{h}_{C_j})$, followed by InfoNCE:

$$\mathcal{L}_{\text{SA-info}} = -\log \frac{\exp(s_{\mathrm{sa}})}{\sum_{j \in \mathcal{B}} \exp(s_{\mathrm{sa}_j})}. \tag{4}$$

Training-Serving Consistency. The causal mask ensures that while historical tokens can attend to the target during training, $\mathbf{C}^*$ is masked out during inference, preventing information leakage and maintaining strict training-serving consistency.

3.2.3 Optimized Training via Decomposed Causal Self-Attention. While the generator architecture provides target-aware modeling capabilities, the conventional approach of concatenating user sequences with candidate items for self-attention results in $O((L_s + B)^2 d)$ complexity, creating a computational bottleneck during training.

We address this through a mathematically equivalent decomposition that maintains causal integrity while enabling efficient parallel computation.

Traditional Approach The baseline computes full self-attention on the concatenated sequence of user history and candidate items:

$$\mathbf{S} = [\mathbf{X}; \mathbf{C}], \quad \mathbf{A} = \mathrm{softmax}\left(\frac{\mathbf{S}\mathbf{W}_Q \mathbf{W}_K^\top \mathbf{S}^\top}{\sqrt{d}} \odot \mathbf{M}\right). \tag{5}$$

Optimized Decomposition Our method decouples attention computation into three coordinated components:

1. User Sequence Self-Attention:

$$\mathbf{Q}_s = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K}_s = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V}_s = \mathbf{X}\mathbf{W}_V \tag{6}$$

$$\mathbf{A}_{ss} = \mathrm{softmax}\left(\frac{\mathbf{Q}_s \mathbf{K}_s^\top}{\sqrt{d}} \odot \mathbf{M}_s\right), \quad \mathbf{O}_s = \mathbf{A}_s \mathbf{V}_s \tag{7}$$

2. Candidate Attention Decomposition:

$$\mathbf{Q}_c = \mathbf{C}\mathbf{W}_Q, \quad \mathbf{K}_c = \mathbf{C}\mathbf{W}_K, \quad \mathbf{V}_c = \mathbf{C}\mathbf{W}_V \tag{8}$$

Cross-Scores (Candidates → User Sequence):

$$\mathbf{A}_{\text{cs}} = \frac{\mathbf{Q}_c \mathbf{K}_X^\top}{\sqrt{d}} \in \mathbb{R}^{B \times L_s} \tag{9}$$

Candidate Self-Correlation Scores:

$$\mathbf{A}_{\text{cc}} = \frac{\sum_{i=1}^{d} \mathbf{Q}_{c,:,i} \odot \mathbf{K}_{c,:,i}}{\sqrt{d}} \in \mathbb{R}^{B \times 1} \tag{10}$$

3. Attention Combination and Output:

$$\mathbf{A}_{\text{combined}} = \text{softmax}\left([\mathbf{A}_{\text{cs}}, \mathbf{A}_{\text{cc}}]\right), \quad \mathbf{V}_{\text{combined}} = [\mathbf{V}_v; \mathbf{V}_c] \tag{11}$$

$$\mathbf{O}_c = \mathbf{A}_{\text{combined}} \mathbf{V}_{\text{combined}}, \quad \mathbf{O}_{\text{final}} = [\mathbf{O}_s; \mathbf{O}_c] \tag{12}$$

Complexity Analysis This decomposition reduces complexity from $O((L_s + B)^2 d)$ to $O(L_s^2 d + BL_s d + Bd)$, achieving 82% FLOPs reduction under typical configurations ($B = 300$, $L_s = 64$, $d = 128$) while maintaining mathematical equivalence with the traditional approach.

The mathematical equivalence proof and detailed algorithm analysis are provided in Appendix A.

3.2.4 Ranker Module. **Encoding Long-Term Context and Candidates:** It processes an extended user history (up to 1000 items) through an MLP to capture long-term preferences $\mathbf{H}_u^{\text{long}} \in \mathbb{R}^{L_l \times d}$. Each candidate item is similarly transformed via a separate MLP to obtain an enriched query representation $\check{\mathbf{x}}^* \in \mathbb{R}^d$.

Lightweight Cross-Attention: For each candidate, a single-head cross-attention mechanism uses the candidate as query and $\mathbf{H}_u^{\text{long}}$ as keys/values, enabling fine-grained, target-aware interaction. The output is projected to a relevance score s_{ca}.

3.2.5 Ranker Module. The Ranker's role is to perform precise, candidate-specific scoring on the small candidate set from the Generator. Its design prioritizes effectiveness within a low compute budget.

Encoding Long-Term Context and Candidates: It processes an extended user history (up to 1000 items) through an MLP to capture long-term preferences $\mathbf{H}_u^{\text{long}} \in \mathbb{R}^{L_l \times d}$. Each candidate item is similarly transformed via a separate MLP to obtain an enriched query representation $\check{\mathbf{x}}^* \in \mathbb{R}^d$.

Cross-Attention Scoring. We then apply a single-head cross-attention mechanism where the candidate serves as query and the historical sequence provides keys and values:

$$\mathbf{z} = \text{Attention}(\mathbf{Q} = \check{\mathbf{x}}^* \mathbf{W}_Q, \mathbf{K} = \mathbf{H}_u^{\text{long}} \mathbf{W}_K, \mathbf{V} = \mathbf{H}_u^{\text{long}} \mathbf{W}_V) \tag{13}$$

$$= \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}. \tag{14}$$

This allows the model to attend to historically relevant items when scoring the candidate. The context vector $\mathbf{z}$ is then projected through an MLP to a relevance score s_{ca}.

Optimization: The Ranker is trained with an InfoNCE loss $\mathcal{L}_{\text{CA-info}}$ using in-batch negatives. Its simplicity keeps inference latency minimal, as it only runs on hundreds (not millions) of items:

$$\mathcal{L}_{\text{CA-info}} = -\log \frac{\exp(s_{\text{ca}})}{\sum_{j \in \mathcal{B}} \exp(s_{\text{ca}_j})}. \tag{15}$$

3.2.6 Multi-Task Training. The Generator (with Auxiliary) and Ranker are trained jointly end-to-end:

$$\mathcal{L}_{\text{total}} = \lambda_0 \mathcal{L}_{\text{NTP}} + \lambda_1 \mathcal{L}_{\text{SA-info}} + \lambda_2 \mathcal{L}_{\text{CA-info}}, \tag{16}$$

where weights λ_i are tuned on a validation set. This ensures both stages are co-adapted: the Generator learns to produce candidates that the Ranker can accurately discriminate.

3.3 Online Inference

Online inference adopts a two-stage latency-sensitive cascade as shown in Figure 2.

Stage-1: NTP Generator. The vector $\mathbf{H}_{NTP}$ is used as a query for MIPS over a quantized embedding index, retrieving a top-$k_1 = 2{,}000$ candidate set C_1 in $O(\log |\mathcal{I}|)$ time.

Stage-2: Ranker. For each $i \in C_1$, we compute the cross-attention score s_{CA} and rerank the candidates to obtain the final top-$k_2 = 500$ items. This computation is purely feedforward and highly parallelizable, enabling high-throughput, low-latency inference when batched on GPU—making it well-suited for real-time serving (see §4.3 for detailed benchmarks).

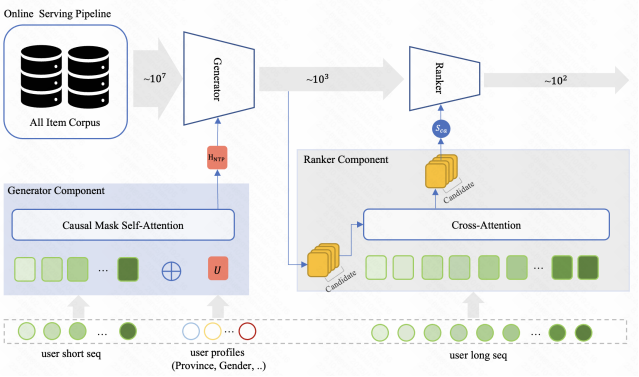


Figure 2: GRank Serving Architecture. Stage-1 (Generator): Leverages user latent interests to generate a personalized candidate subset. Starge-2 (Ranker):Employs a target-aware cross-attention scorer for fine-grained relevance estimation.

4 Experiments

We conducted rigorous experiments to answer five research questions (RQs).

- **RQ1**: How effective is GRank compared with state-of-the-art models in the industry?
- **RQ2**: Does the GRank architecture meet industrial latency constraints?
- **RQ3**: What is the contribution of each core architectural component to the overall performance?
- **RQ4**: How do hyper-parameter selections enable the optimal trade-off between efficiency and performance in industrial deployment?
- **RQ5**: Does GRank bring significant online gains in our short-video service?

Table 1: Performance Comparison of SOTA Methods on Different Datasets. The best and second-best results highlighted in bold font and underlined. *Note: TDM was only evaluated on the Industry Dataset due to its deployment complexity, using our existing infrastructure.*

Method	User Behavior		MovieLens		Industry Datasets	
	Recall@50	NDCG@50	Recall@50	NDCG@50	Recall@500	NDCG@500
DSSM	0.2711	0.1911	0.1940	0.0792	0.1068	0.0360
Kuaiformer	0.3270	0.2347	0.2538	0.0906	0.1151	0.0386
TDM	-	-	-	-	0.1766	0.0535
NANN	0.3264	0.1765	0.2633	0.1017	0.1326	0.0466
Streaming VQ	0.3220	0.2262	0.2554	0.0907	0.1296	0.0603
GRank	**0.4348**	**0.2780**	**0.3350**	**0.1250**	**0.2346**	**0.0775**
Relative Gain	+33.0%	+18.4%	+27.2%	+22.9%	+32.8%	+28.5%

4.1 Experimental Setup

4.1.1 Datasets and Evaluation Protocol. We evaluate GRank on one industrial and two public datasets (Table 2): **MovieLens-20M** (movie recommendations), **Taobao UserBehavior** (e-commerce), and **Kuaishou Industrial** (short-video platform). All datasets are split chronologically: Kuaishou uses 6 days for training and the next day for testing, while the public datasets follow standard 90%/10% split.

Table 2: Dataset statistics after preprocessing

	UserBehavior	MovieLens-20M	Kuaishou
Users	964K	138K	108M
Items	4.2M	27K	42M
Interactions	1.7M	9.3M	4B
Avg. Seq. Len.	101	144	980

4.1.2 Evaluation Metrics and Baselines. We report **Recall@K** and **NDCG@K** (retrieval from full corpus), and **QPS** (maximum throughput under P99 latency $\leq$ 100ms).

Baselines represent key retrieval paradigms: **DSSM** (dual-tower), **Kuaiformer** (generative), **TDM** (tree-based), **NANN** (graph-based), and **StreamingVQ** (quantization). All use identical training data, embeddings ($d = 128$), and hardware configurations.

4.1.3 Implementation Details. GRank uses embedding dimension $d = 128$, 4-layer causal self-attention, 1-layer cross-attention, sequence lengths of 64 (short-term) and 1000 (long-term), candidate set size 2000. All baselines use identical embeddings and hardware for fair comparison. Comprehensive training, optimization, and deployment details are provided in Appendix C.

4.2 Offline Evaluation (RQ1)

As summarized in Table 1, GRank achieves state-of-the-art performance across all datasets, demonstrating the effectiveness of its unified, index-free paradigm.

Our results demonstrate two key findings: First, GRank significantly outperforms target-agnostic models including generative (Kuaiformer) and dual-tower (DSSM) approaches, improving Recall@50 by **33.0%** on the User Behavior dataset. All target-aware methods (TDM, NANN, StreamingVQ) consistently outperform those baselines on industrial data, confirming the necessity of target-aware modeling.

Second, GRank achieves superior performance even compared to sophisticated target-aware systems that rely on structured indices, with **32.8%** higher Recall@500 than tree-based TDM on the Industry Dataset. The fact that GRank attains this superior accuracy without any structured index highlights the sufficiency of a well-trained generator combined with a lightweight ranker.

These consistent gains across diverse datasets confirm GRank's effectiveness in bridging the accuracy-efficiency gap in large-scale retrieval.

4.3 Online Service Performance (RQ2)

This section evaluates whether GRank's two-stage architecture meets industrial latency constraints (RQ2). All experiments used production servers with identical GPU configurations, measuring QPS under the P99 latency constraint of $\leq$ 100 ms. Reported latencies include the full pipeline from feature extraction to final scoring.

As shown in Table 3, GRank achieves a QPS of 767, significantly outperforming NANN and TDM while maintaining superior recall. The suboptimal throughput of NANN and TDM can be attributed to path uncertainty, candidate set size fluctuations, and multi-round retrieval overhead. In contrast, GRank's efficiency stems from its two-stage design: the generator rapidly reduces candidate sets via efficient MIPS, avoiding complex tree/graph traversals, while the lightweight ranker applies precise target-aware scoring with minimal overhead. This approach avoids the latency variability of structured indices while delivering higher accuracy than target-agnostic models.

4.4 Ablation Study on Architectural Components (RQ3)

Our ablation study addresses three pivotal questions through corresponding model variants:

- **Pure Generator**: How does the generator perform alone as a retrieval baseline? — Uses only the first-stage generator without ranker.

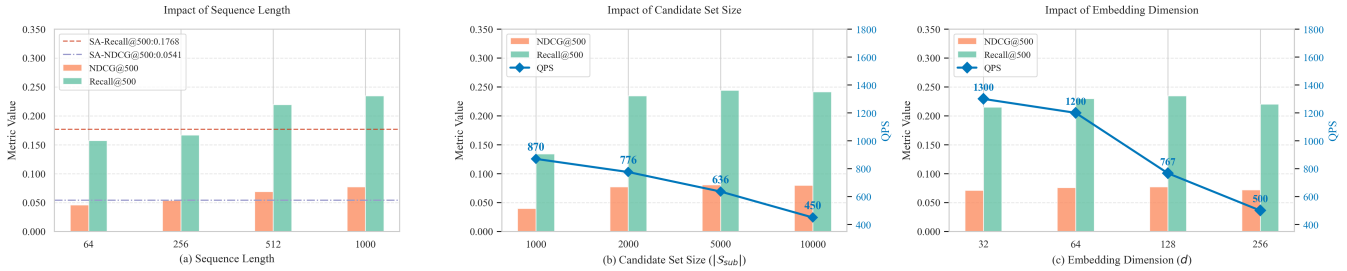


Figure 3: Hyper-parameter analysis on the efficiency-performance trade-off. (a) Sequence length: CA Recall@500 increases monotonically with L, peaking when $L = 1\,000$, significantly outperforming the SA module while maintaining lower latency. (b) Candidate size: diminishing returns begin at $k_1 = 2\,000$; recall improves by only 0.1 pp at $k_1 = 5\,000$ while QPS drops 18%, making 2 000 the pragmatic optimum. (c) Top embedding dimension: recall saturates at $d_{\text{top}} = 128$ and falls at 256 owing to memory pressure; $d_{\text{top}} = 64$ offers a high-ROI fallback, trading 3% recall for 56% QPS gain.

Table 3: End-to-end latency, throughput and recall comparison. SLA threshold is 100ms. All latency values include full retrieval service pipeline (feature processing + core retrieval logic).

Method	Recall @500	NDCG @500	QPS	P50 Lat. (ms)	P99 Lat. (ms)
DSSM	0.1068	0.0360	1300	50	61
Kuaiformer	0.1151	0.0386	772	59	87
TDM	0.1766	0.0535	273	55	96
NANN	0.1326	0.0466	384	70	100
StreamVQ	0.1296	0.0603	450	48	75
GRank	**0.2346**	**0.0775**	**767**	**48**	**73**

- **Gen+SA**: Does self-attention provide sufficient performance gains? — Generator with 4-layer self-attention only.
- **Gen+CA**: Can cross-attention offer a better efficiency-accuracy trade-off? — Generator with cross-attention only.

The full GRank model then answers: can a hybrid architecture synergize both to achieve the optimal balance?

Table 4: Ablation Results of Hybrid Attention Architecture

Config	Generator Recall@2000	Recall @500	NDCG @500	QPS
Pure Generator	0.1512	0.1190	0.0405	1333
Gen+SA Rank	0.3427	0.1768	0.0541	340
Gen+CA Rank	0.1820	0.1363	0.0502	767
Full GRank	**0.3685**	**0.2346**	**0.0775**	**767**

The ablation results conclusively demonstrate that our two-stage design with hybrid attention mechanisms achieves the best trade-off between retrieval quality and computational efficiency.

- **Universal Offline-Enhancement Paradigm for Generator**. To further elucidate the performance gains of GRank, we dissect the behaviour of the Generator itself. Since the 2 000 candidates produced in the first stage form the *absolute* ceiling for any subsequent ranker, the jump of Generator Recall@2000 from 0.1512 to 0.3685 *significantly contributes* to the final Recall@500 of 0.2346. This observation reveals that injecting a target-aware ranking loss during training substantially strengthens the user representation of a dual-tower model, while leaving the tower architecture *completely untouched* at inference—introducing **zero** additional latency. Consequently, the "**offline optimise, transparent deploy**" paradigm acts as a drop-in, model-agnostic performance booster that is instantly applicable to *any* representation-based or dual-tower retrieval system, delivering higher accuracy with no extra parameters, no extra engineering, and no extra cost at serving time.
- **Second-Stage Rescoring is Critical for Accuracy Gains**. The removal of rescoring drastically impairs performance (Recall@500: -50.7%), highlighting its necessity. All rescoring variants deliver substantial metric gains over the pure generator, confirming the value of second-stage refinement, though at the cost of reduced QPS. The full GRank model achieves the most favorable accuracy-efficiency trade-off.
- **CA-SA Synergy Outperforms Isolated Pathways**.Our ablation shows that combining CA and SA outperforms either approach alone, with each mechanism offering distinct advantages:
  - **Generator with Self-Attention (Gen+SA)**: This configuration achieves a Recall@500 of 0.1768, representing a significant improvement over the pure generator baseline. However, the quadratic computational complexity ($O(4 * n^2)$) of 4 layers self-attention leads to substantial latency overhead.
  - **Generator with Cross-Attention (Gen+CA)**: This approach also demonstrates notable improvement in Recall@500 (0.1363) over the baseline. More importantly, CA's linear complexity ($O(n)$) enables efficient processing, particularly advantageous for long-sequence modeling tasks. Nevertheless, its accuracy remains inferior to the SA-enhanced variant.
  - **Hybrid CA-SA synergy optimizes trade-offs:** The complete GRank framework leverages both attention mechanisms: CA for efficient global modeling and SA for capturing fine-grained sequential dependencies during training and influence the update dynamics of shared sparse representations. Crucially,

Table 5: Online A/B Test Results of GRank Across Different Scenarios

Applications	Overall App Metrics				Recall Pathway Metrics		
	Total App Usage Time	Usage Time per User	Video Watch Time	Effective Interests	Show Ratio	Avg. Watch Time	UV Coverage
Kuaishou Single Page	+0.160%	+0.139%	+0.347%	+0.527%	+21.92%	+16.26%	+16.50%
Kuaishou Lite Single Page	+0.165%	+0.118%	+0.233%	+0.384%	+20.31%	+11.69%	+12.29%

SA is excluded from inference, preserving the latency benefits of CA while maintaining superior accuracy.

4.5 Balancing Efficiency and Performance through Hyper-Parameter Selection (RQ4)

We systematically examine how hyper-parameters govern the trade-off between efficiency and performance in industrial deployment. Rather than merely reporting sensitivity, we identify optimal configurations that achieve the best practical balance for our production environment. Figure 3 presents the hyper-parameter analysis, with detailed numerical data provided in Appendix B (Tables 6 – 8).

4.5.1 Sequence Length: Long-Range Modeling with Linear Complexity. The analysis focuses on the CA module's sequence length. To maximize efficiency, scoring is performed exclusively by the CA module during inference. As shown in Figure 3.a, the performance improves consistently with increasing L: when $L = 1,000$, Recall@500 reaches 0.2345, significantly surpassing the SA module's performance(0.1768) while maintaining substantially lower latency. This demonstrates that increasing sequence length effectively compensates for excluding the SA module at inference, achieving an optimal balance where enhanced effectiveness meets practical efficiency constraints.

4.5.2 Candidate Size: ROI-Optimal Tuning. The generator candidate size k_1 balances recall and throughput (Fig. 3.b). For our latency budget, $k_1 = 2,000$ is empirically optimal: beyond this, recall gains diminish sharply (only 4.27% at $k_1 = 5,000$) while QPS drops 18%, confirming its ROI-optimal status. This aligns with GRank's two-stage design—the Generator establishes a high-recall ceiling, leaving fine-grained discrimination to the Ranker. Fig. 3.b provides a general performance–latency tradeoff curve; a practical tuning starting point is $k_1 = 2 \times n_{\text{downstream}}$, adjustable for recall or latency priorities.

4.5.3 Embedding Dimension: Performance Saturation under Memory Constraints. We evaluate the selection of the top embedding dimension d_{top} under production memory constraints. As shown in Figure 3.c, recall increases monotonically with increasing d_{top} and saturates at 128, but drops significantly at 256 due to memory constraints that limit candidate pool size. Meanwhile, $d_{\text{top}} = 64$ maintains comparable performance while significantly improving system throughput. Therefore, $d_{\text{top}} = 128$ is optimal for accuracy-critical scenarios, while $d_{\text{top}} = 64$ provides an ideal alternative for throughput-sensitive deployments requiring efficiency trade-offs.

4.6 Online Results (RQ5)

We conducted a one-week A/B test on Kuaishou's single-column recommendation platforms by replacing the original Kuaiformer recall with GRank while maintaining identical experimental conditions. The results in Table 5 demonstrate GRank's effectiveness across both business and technical dimensions.

Overall App Performance. GRank delivers consistent improvements in core user engagement metrics, with particular strength in video consumption and preference modeling. The framework effectively translates retrieval enhancements into measurable business value, with both main and lite versions showing positive trends across all key indicators.

Recall Quality. GRank improves recall effectiveness, with higher show ratio, UV coverage, and average watch time per session, reflecting improved content relevance and increased user reach. These consistent improvements demonstrate GRank's ability to enhance the entire recommendation ecosystem through superior retrieval.

The correlated improvements across both business metrics and technical indicators establish that GRank's architectural innovations directly translate to enhanced user experience and platform value.

5 Conclusion

In this paper, we presented **GRank**, a novel retrieval framework that fundamentally redefines the balance between efficiency and precision in large-scale recommendation. By decomposing the retrieval process into a decoupled **Generate-then-Rank** paradigm, GRank effectively bridges the gap between target-agnostic efficiency and target-aware expressiveness. Our approach demonstrates that high-precision, candidate-specific interaction modeling can be achieved without the rigid constraints and operational burdens of structured indices, such as trees or graphs.

Extensive experiments demonstrate over 30% improvement in Recall, validating the framework's effectiveness. GRank has been deployed in production since Q2 2025, now serving 400 million users and handling 50 billion daily requests, confirming its scalability and real-world impact. Ultimately, GRank provides a scalable and index-free alternative for modern retrieval systems, paving the way for more flexible and intelligent candidate generation in the industry.

References

[1] Xingyan Bin, Jianfei Cui, Wujie Yan, Zhichen Zhao, Xintian Han, Chongyang Yan, Feng Zhang, Xun Zhou, Qi Wu, and Zuotao Liu. 2025. Real-time Indexing for Large-scale Recommendation by Streaming Vector Quantization Retriever. *arXiv preprint arXiv:2501.08695* (2025).

[2] Fedor Borisyuk, Qingquan Song, Mingzhou Zhou, Ganesh Parameswaran, Madhu Arun, Siva Popuri, Tugrul Bingol, Zhuotao Pei, Kuang-Hsuan Lee, Lu Zheng, et al.

2024. LiNR: Model Based Neural Retrieval on GPUs at LinkedIn. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 4366–4373.
[3] Rihan Chen, Bin Liu, Han Zhu, Yaoxuan Wang, Qi Li, Buting Ma, Qingbo Hua, Jun Jiang, Yunlong Xu, Hongbo Deng, and Bo Zheng. 2022. Approximate Nearest Neighbor Search under Neural Similarity Metric for Large-Scale Recommendation. arXiv:2202.10226 [cs.IR] https://arxiv.org/abs/2202.10226
[4] Chih-Yi Chiu, Amorntip Prayoonwong, and Yin-Chih Liao. 2019. Learning to index for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 42, 8 (2019), 1942–1956.
[5] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.
[6] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.
[7] Weihao Gao, Xiangjun Fan, Chong Wang, Jiankai Sun, Kai Jia, Wenzhi Xiao, Ruofan Ding, Xingyan Bin, Hui Yang, and Xiaobing Liu. 2021. Deep Retrieval: Learning A Retrievable Structure for Large-Scale Recommendations. arXiv:2007.07203 [cs.IR] https://arxiv.org/abs/2007.07203
[8] Chengcheng Guo, Junda She, Kuo Cai, Shiyao Wang, Qigen Hu, Qiang Luo, Kun Gai, and Guorui Zhou. 2025. MISS: Multi-Modal Tree Indexing and Searching with Lifelong Sequential Behavior for Retrieval Recommendation. arXiv:2508.14515 [cs.IR] https://arxiv.org/abs/2508.14515
[9] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. arXiv:1908.10396 [cs.LG] https://arxiv.org/abs/1908.10396
[10] Po-Sen Huang, Xiaodong He, Jianfeng Gao, Li Deng, Alex Acero, and Larry Heck. 2013. Learning deep structured semantic models for web search using clickthrough data. In *Proceedings of the 22nd ACM international conference on Information & Knowledge Management*. 2333–2338.
[11] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 33, 1 (2010), 117–128.
[12] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
[13] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
[14] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2017. Billion-scale similarity search with GPUs. arXiv:1702.08734 [cs.CV] https://arxiv.org/abs/1702.08734
[15] Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. arXiv:1808.09781 [cs.IR] https://arxiv.org/abs/1808.09781
[16] Chao Li, Zhiyuan Liu, Mengmeng Wu, Yuchi Xu, Pipei Huang, Huan Zhao, Guoliang Kang, Qiwei Chen, Wei Li, and Dik Lun Lee. 2019. Multi-Interest Network with Dynamic Routing for Recommendation at Tmall. arXiv:1904.08030 [cs.IR] https://arxiv.org/abs/1904.08030
[17] Chi Liu, Jiangxia Cao, Rui Huang, Kai Zheng, Qiang Luo, Kun Gai, and Guorui Zhou. 2024. KuaiFormer: Transformer-Based Retrieval at Kuaishou. *arXiv preprint arXiv:2411.10057* (2024).
[18] Yu. A. Malkov and D. A. Yashunin. 2018. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. arXiv:1603.09320 [cs.DS] https://arxiv.org/abs/1603.09320
[19] Aleksandr V. Petrov and Craig Macdonald. 2023. Generative Sequential Recommendation with GPTRec. arXiv:2306.11114 [cs.IR] https://arxiv.org/abs/2306.11114
[20] Steffen Rendle, Li Zhang, and Yehuda Koren. 2019. On the difficulty of evaluating baselines: A study on recommender systems. *arXiv preprint arXiv:1905.01395* (2019).
[21] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. arXiv:1904.06690 [cs.IR] https://arxiv.org/abs/1904.06690
[22] Yijia Sun, Shanshan Huang, Linxiao Che, Haitao Lu, Qiang Luo, Kun Gai, and Guorui Zhou. 2025. MPFormer: Adaptive Framework for Industrial Multi-Task Personalized Sequential Retriever. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 2832–2841.
[23] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1059–1068.
[24] Rui Zhou, Qinglin Jia, Bo Chen, Peng Xu, Yijia Sun, Siyuan Lou, Chaoxin Fu, Mengyuan Fu, Guoming Shen, Zheli Zhou, Jinlong Jiao, Naifu Zhou, Shijie Guan, Yunjing Qi, Shiyao Wang, Xinchen Luo, Qigen Hu, Chaoyi Ma, Xiao Lv, Qiang Luo, Yuyang Ye, Luankang Zhang, Defu Lian, Ruiming Tang, Guorui Zhou, Han Li, Kun Gai, Hao Wang, and Enhong Chen. 2026. A Survey of User Lifelong Behavior Modeling: Perspectives on Efficiency and Effectiveness. *Preprints* (January 2026). doi:10.20944/preprints202601.1559.v1
[25] Han Zhu, Daqing Chang, Ziru Xu, Pengye Zhang, Xiang Li, Jie He, Han Li, Jian Xu, and Kun Gai. 2019. Joint Optimization of Tree-based Index and Deep Model for Recommender Systems. arXiv:1902.07565 [cs.IR] https://arxiv.org/abs/1902.07565
[26] Han Zhu, Xiang Li, Pengye Zhang, Guozheng Li, Jie He, Han Li, and Kun Gai. 2018. Learning Tree-based Deep Model for Recommender Systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery Data Mining*. ACM, 1079–1088. doi:10.1145/3219819.3219826
[27] Han Zhu, Xiang Li, Pengye Zhang, Guozheng Li, Jie He, Han Li, and Kun Gai. 2018. Learning tree-based deep model for recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1079–1088.
[28] Jingwei Zhuo, Ziru Xu, Wei Dai, Han Zhu, Han Li, Jian Xu, and Kun Gai. 2020. Learning Optimal Tree Models Under Beam Search. arXiv:2006.15408 [stat.ML] https://arxiv.org/abs/2006.15408

A Mathematical Equivalence of Decomposed Causal Self-Attention

This appendix provides the formal mathematical foundation for the decomposed causal self-attention mechanism described in Section 3.2.3. Here we use consistent notation with the main text: L denotes historical sequence length, B denotes candidate batch size, and d denotes hidden dimension.

The decomposition presented in Section 3.2.3 splits the input into historical ($\mathbf{X} \in \mathbb{R}^{L\times d}$) and candidate ($\mathbf{C} \in \mathbb{R}^{B\times d}$) components. In this appendix, we prove that this decomposition is mathematicaccy equivalent to standard causal attention.

A.1 Notation and Preliminaries

Let $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{L\times d}$ be the query, key, and value matrices, respectively, where L is the sequence length and d is the hidden dimension. The standard scaled dot-product attention is defined as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V} \tag{17}$$

For causal self-attention, we apply a causal mask $\mathbf{M} \in \{0, -\infty\}^{L\times L}$ where $M_{ij} = 0$ if $i \geq j$ and $M_{ij} = -\infty$ otherwise:

$$\text{CausalAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{M}\right)\mathbf{V} \tag{18}$$

A.2 Decomposition of Causal Self-Attention

Our decomposed attention splits the sequence into two parts: historical items (S, length L_s) and candidate items (C, length L_c), such that $L = L_s + L_c$. Let:

$$\mathbf{Q} = \begin{bmatrix}\mathbf{Q}_s\\ \mathbf{Q}_c\end{bmatrix}, \qquad \mathbf{K} = \begin{bmatrix}\mathbf{K}_s\\ \mathbf{K}_c\end{bmatrix}, \qquad \mathbf{V} = \begin{bmatrix}\mathbf{V}_s\\ \mathbf{V}_c\end{bmatrix} \tag{19}$$

where $\mathbf{Q}_s, \mathbf{K}_s, \mathbf{V}_s \in \mathbb{R}^{L_s\times d}$ and $\mathbf{Q}_c, \mathbf{K}_c, \mathbf{V}_c \in \mathbb{R}^{L_c\times d}$.

The attention matrix before softmax can be partitioned as:

$$\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} = \frac{1}{\sqrt{d}}\begin{bmatrix}\mathbf{Q}_s\mathbf{K}_s^\top & \mathbf{Q}_s\mathbf{K}_c^\top\\ \mathbf{Q}_c\mathbf{K}_s^\top & \mathbf{Q}_c\mathbf{K}_c^\top\end{bmatrix} \tag{20}$$

Applying the causal mask $\mathbf{M}$ yields:

$$\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{M} = \frac{1}{\sqrt{d}}\begin{bmatrix}\mathbf{Q}_s\mathbf{K}_s^\top + \mathbf{M}_s & \mathbf{0}\\ \mathbf{Q}_c\mathbf{K}_s^\top & \mathbf{Q}_c\mathbf{K}_c^\top\end{bmatrix} \tag{21}$$

where $\mathbf{M}_s$ is the causal mask for the historical subsequence, and the zero block enforces that historical items cannot attend to future candidates.

A.3 Mathematical Equivalence Proof

Theorem 1. The decomposed causal self-attention computes the exact same output as the standard causal self-attention.

Proof. From Equation (21), the softmax operation can be applied block-wise. Let:

$$\mathbf{A}_{ss} = \text{softmax}\left(\frac{\mathbf{Q}_s\mathbf{K}_s^\top}{\sqrt{d}} + \mathbf{M}_s\right) \tag{22}$$

$$\mathbf{A}_{cs} = \text{softmax}\left(\frac{\mathbf{Q}_c\mathbf{K}_s^\top}{\sqrt{d}}\right) \tag{23}$$

$$\mathbf{A}_{cc} = \text{softmax}\left(\frac{\mathbf{Q}_c\mathbf{K}_c^\top}{\sqrt{d}}\right) \tag{24}$$

The fucc attention output is:

$$\text{CausalAttention}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \begin{bmatrix}\mathbf{A}_{ss} & \mathbf{0}\\ \mathbf{A}_{cs} & \mathbf{A}_{cc}\end{bmatrix}\begin{bmatrix}\mathbf{V}_s\\ \mathbf{V}_c\end{bmatrix} \tag{25}$$

$$= \begin{bmatrix}\mathbf{A}_{ss}\mathbf{V}_s\\ \mathbf{A}_{cs}\mathbf{V}_s + \mathbf{A}_{cc}\mathbf{V}_c\end{bmatrix} \tag{26}$$

Our decomposed implementation computes:

$$\text{Historical output: } \mathbf{O}_s = \mathbf{A}_{ss}\mathbf{V}_s \tag{27}$$

$$\text{Candidate-to-historical: } \mathbf{C}_{cs} = \mathbf{A}_{cs}\mathbf{V}_s \tag{28}$$

$$\text{Candidate self-attention: } \mathbf{C}_{cc} = \mathbf{A}_{cc}\mathbf{V}_c \tag{29}$$

$$\text{Final candidate output: } \mathbf{O}_c = \mathbf{C}_{cs} + \mathbf{C}_{cc} \tag{30}$$

Concatenating $\mathbf{O}_s$ and $\mathbf{O}_c$ yields exactly Equation (26). The computational advantage comes from: (1) $\mathbf{A}_{cc}$ is a diagonal matrix due to the causal constraint between candidates, accowing $O(L_c)$ computation; and (2) $\mathbf{A}_{cs}$ has shape $L_c \times L_s$, avoiding the $O(L_c^2)$ computation of fucc candidate-to-candidate attention. □

B Detailed Hyper-parameter Analysis Data

This appendix provides the detailed numerical data corresponding to the hyper-parameter analysis shown in Figure 3 in the main text.

Table 6: CA Recall@500 and Latency vs. Sequence Length (L)

Sequence Length (L)	Recall@500	NDCG@500
64(SA)	0.1768	0.0541
64	0.1574	0.0459
256	0.1670	0.0539
512	0.2193	0.0692
1,000(ours)	**0.2346**	**0.0775**

Note: The peak CA Recall@500 is achieved at $L = 1,000$. Latency increases approximately linearly with sequence length.

Table 7: CA Recall@500 and QPS vs. Candidate Size (k_1)

Candidate Size (k_1)	Recall@500	NDCG@500	QPS
1,000	0.1345	0.0398	879
2,000(ours)	0.2346	0.0775	776
5,000	0.2419	0.0800	636
10,000	0.2442	0.0806	450

Note: Diminishing returns begin at $k_1 = 2,000$, where recall improves only marginally (0.1 percentage points at $k_1 = 5,000$) while QPS drops significantly (18% from $k_1 = 2,000$ to $5,000$).

Table 8: CA Recall@500 and QPS vs. Top Embedding Dimension (d_{top})

Top Embedding Dim (d_{top})	Recall@500	NDCG@500	QPS
32	0.0711	0.2150	1300
64	0.0758	0.2299	1200
128(ours)	0.0775	0.2346	767
256	0.0722	0.2202	500

Note: Recall saturates at $d_{\text{top}} = 128$ and declines at 256 due to memory pressure. Using $d_{\text{top}} = 64$ trades approximately 3% recall for a 56% QPS gain compared to $d_{\text{top}} = 128$.

C Training and Implementation Details

This appendix provides the complete training, optimization, and deployment configurations for the GRank framework, supplementing the high-level description in Section 4.1.3. These details are critical for reproducibility and industrial adoption.

C.1 Training Data & Sampling Strategy

We adopt a session-based training paradigm. For each positive user-item pair (u, i^+), we use the user's chronological behavior sequence ending just before i^+'s timestamp to construct the model input, preventing any future data leakage. We employ efficient *in-batch negative sampling*: within each mini-batch of 300 examples, items from all other users serve as negatives for the current user. This yields approximately 299 negatives per positive example at minimal computational cost, facilitating effective contrastive learning.

C.2 Optimization Configuration

The model is trained end-to-end using the AdamW optimizer with a learning rate of 1×10^{-3}, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and a weight decay of 0.01. Gradient clipping is applied at a global norm of 10.0.

C.3 Hardware & Training Scale

Training was conducted on a distributed cluster with 2 GPUs, leveraging automatic mixed precision (AMP) to accelerate computation and optimize memory usage. For the largest dataset (Kuaishou), the total training time was approximately 24 hours.

C.4 Serving Configuration

For online inference, the Generator computes a single user embedding per request, while all item tower embeddings are pre-computed

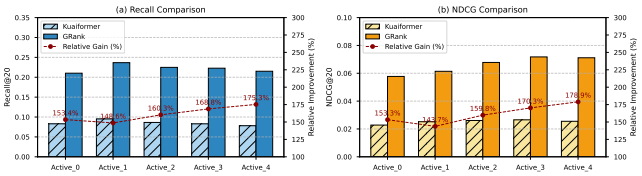
offline. The first-stage retrieval is performed via a fast, approximate k-nearest neighbor (KNN) search over these item tower embeddings to obtain a candidate set. The lightweight Ranker is accelerated using FP16 precision. The complete two-stage pipeline is deployed as a unified service on GPU machines, engineered to meet the industrial P99 latency target of $\leq$ 100ms per request.

D Robustness Analysis Across User and Item Subgroups

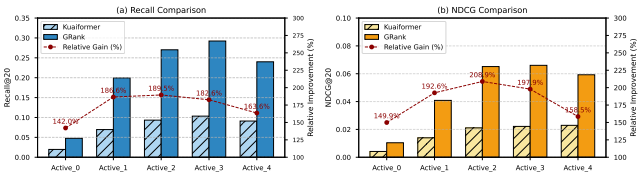
We analyze model robustness under heterogeneous data distributions by evaluating performance across user and item activity subgroups. Users and items are stratified into five groups according to interaction frequency, where *Active_0* denotes the least active group and *Active_4* the most active.

As shown in Table 4a, GRank consistently outperforms Kuaiformer across all user activity groups in terms of both Recall and NDCG. Notably, GRank maintains strong performance for low-activity users, indicating reduced sensitivity to interaction sparsity. In contrast, Kuaiformer exhibits weaker and less stable performance, particularly in sparse regimes.

Similar trends are observed across item activity groups in Table 4a. GRank achieves uniformly higher Recall and NDCG for both low-activity (long-tail) and high-activity items, suggesting improved generalization beyond popular content. Overall, these results demonstrate that GRank is robust to activity-level heterogeneity on both the user and item sides, providing stable recommendation quality under varying data sparsity conditions.



(a) User Activity Groups



(b) Item Activity Groups

Figure 4: Robustness evaluation across stratified user and item activity subgroups. Entities are grouped from *Active_0* (least active users / long-tail items) to *Active_4* (daily active users / popular items). Bars denote absolute metrics (left Y-axis), while red lines represent the relative improvement of GRank over Kuaiformer (right Y-axis). GRank exhibits consistent superiority and strong resilience to interaction sparsity in both user and item dimensions.